

MAXIMUS

DOSSIER TECHNIQUE

Architecture · Fonctionnalités · API · Sécurité · Développement des modules

Version 1.0 · 5 septembre 2026
Environnement de validation : démo MAXIMUS
Backend actif : Laravel + PostgreSQL
Frontend : React + TypeScript + Vite

Document généré à partir de l'état vérifié du projet.
Les secrets et identifiants de démonstration sont volontairement exclus.

Sommaire

- 1. Résumé exécutif
- 2. Périmètre et statut du document
- 3. Architecture générale
- 4. Modèle produit et multi-entreprise
- 5. Catalogue fonctionnel des modules
- 6. Frontend MAXIMUS
- 7. Backend Laravel et API
- 8. Authentification, rôles et permissions
- 9. Persistance PostgreSQL et modèle de données
- 10. Sécurité et isolation multi-entreprise
- 11. Méthode de développement d'un nouveau module
- 12. Tests et validation actuels
- 13. Exploitation et runtime de démonstration
- 14. Limites connues et prochaines étapes
- 15. Glossaire
- 16. Conclusion technique

1. Résumé exécutif

MAXIMUS ERP est une plateforme web modulaire de pilotage des opérations d’entreprise. Un module est un outil métier conçu indépendamment de toute entreprise, puis proposé aux entreprises autorisées. L’entreprise consomme le module ; elle n’en possède pas le code et ne doit jamais devenir une dépendance du module.

L’architecture de validation actuelle repose sur un frontend React/Vite, une API Laravel 12 fonctionnant avec PHP 8.2, PostgreSQL comme base principale et une authentification interne par session HTTP sécurisée. Express reste présent comme ancien code de secours mais n’est pas le backend actif de la démonstration.

Décision	Règle retenue
Produit	MAXIMUS est un produit unique composé de modules métier réutilisables.
Module	Un module est défini globalement, sans dépendre d’une entreprise particulière.
Entreprise	Une entreprise reçoit une autorisation, une configuration et des données propres.
Sécurité	L’entreprise est résolue depuis la session serveur, jamais depuis un identifiant client arbitraire.
Backend actif	Laravel 12 + PHP 8.2 + PostgreSQL.
Frontend	React + TypeScript + Vite, avec routage client Wouter.
Validation	Démo fonctionnelle ; la préparation de production est volontairement différée.

2. Périmètre et statut du document

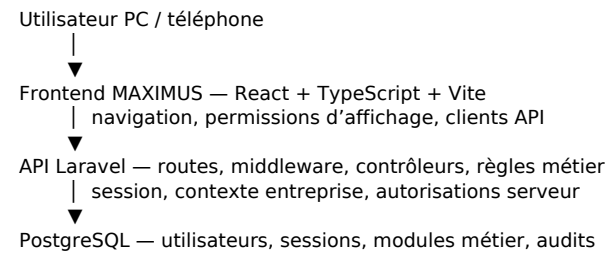
Ce dossier décrit l’état technique vérifié de MAXIMUS au 5 septembre 2026. Il couvre les fonctionnalités présentes dans le frontend et le backend Laravel, les contrats d’architecture, les règles de sécurité et la méthode de développement des prochains modules.

Ce document n’est pas une spécification de production finale. Les zones explicitement marquées « prochaine étape » ne doivent pas être considérées comme déjà livrées.

Statut	Signification
Livré dans la démo	Fonctionnalité visible et/ou couverte par le code et les tests actuels.
Règle d’architecture	Décision à respecter pour tout nouveau développement.
Prochaine étape	Extension nécessaire avant une utilisation production ou multi-client avancée.

3. Architecture générale

3.1 Vue d’ensemble



Le navigateur ne communique pas directement avec PostgreSQL. Il passe par les routes Laravel. Le frontend peut masquer une rubrique, mais cette présentation ne constitue jamais la protection principale : chaque route sensible doit revérifier la session, l’entreprise, le rôle et l’action côté serveur.

3.2 Artifacts et workflows

Élément	Responsabilité
artifacts/maximus	Application web MAXIMUS et interface de gestion.
artifacts/api-server	Serveur API Laravel et code historique Express.
artifacts/mockup-sandbox	Serveur de prévisualisation de composants.
Workflow API Server	Provisionnement de la démo puis serveur PHP intégré sur le port fourni.
Workflow MAXIMUS web	Serveur Vite du frontend.

3.3 Arborescence utile

```
artifacts/  
├── api-server/  
│   ├── laravel/  
│   │   ├── app/Http/Controllers/Api/  
│   │   ├── app/Http/Middleware/  
│   │   ├── app/Models/  
│   │   ├── app/Support/  
│   │   ├── database/migrations/  
│   │   ├── routes/  
│   │   └── tests/Feature/  
│   └── MODULE_GUIDE.md  
├── maximus/  
│   └── src/  
│       ├── components/  
│       ├── lib/  
│       ├── pages/  
│       └── routes/  
└── mockup-sandbox/
```

4. Modèle produit et multi-entreprise

4.1 Le module est global

La définition d'un module existe dans le catalogue MAXIMUS avant toute entreprise. Elle comprend son identifiant stable, son nom, sa description, ses fonctionnalités, ses dépendances et son contrat métier. Elle ne doit pas contenir de nom d'entreprise, de couleur client, de compte utilisateur ou de règle spécifique à KORA.

```
Catalogue global  
├── stocks  
│   ├── products  
│   ├── entries  
│   ├── exits  
│   ├── inventory  
│   └── reports
```

Accès des entreprises

```
├── Entreprise A : stocks activé  
├── Entreprise B : stocks activé avec fonctionnalités limitées  
└── Entreprise C : stocks non autorisé
```

Données

```
└── chaque ligne métier porte le company_id de son entreprise
```

4.2 Trois niveaux à ne pas confondre

Niveau	Question	Exemple
Catalogue global	Quel outil MAXIMUS existe-t-il ?	Module Stocks, identifiant stocks.
Autorisation entreprise	Cette entreprise peut-elle utiliser cet outil ?	Stocks activé pour l'entreprise A.
Permission acteur	Cet employé peut-il exécuter cette action ?	Magasinier : voir et créer des articles.
Donnée tenant	À quelle entreprise appartient cette donnée ?	Produit stocké avec company_id = A.

4.3 Règle de traitement d'une requête

1. Authentifier la session MAXIMUS.
2. Résoudre l'acteur connecté et son entreprise côté serveur.
3. Vérifier que l'entreprise est autorisée à utiliser le module.
4. Vérifier le rôle, l'unité, la sous-fonctionnalité et l'action.
5. Filtrer les lectures et écritures par le contexte entreprise résolu.
6. Tracer l'opération lorsqu'elle constitue une décision ou une modification sensible.

5. Catalogue fonctionnel des modules

Le catalogue frontend actuel contient les modules métier suivants. Certains sont déjà représentés par des écrans fonctionnels, tandis que d'autres disposent encore d'un espace de travail générique ou d'une base de données de démonstration.

Identifiant	Module	Fonctionnalités principales
commerce	Gestion commerciale	Clients, devis et commandes, chiffre d'affaires.
ventes	Ventes	Devis, commandes, facturation.
achats	Achats	Demandes d'achat, commandes fournisseurs, réceptions.
stocks	Gestion de stock	Articles, entrées/sorties, alertes de seuil.
finance	Finance	Suivi des paiements, trésorerie, rapports financiers.
comptabilite	Comptabilité	Plan comptable, journaux, rapprochement.
rh	Ressources humaines	Employés, rôles, organisation.
presences	Présences	Pointage, historique, rapports.
paie	Paie	Périodes de paie, bulletins, déclarations.
crm	CRM / Clients	Fiches clients, opportunités, relances.
fournisseurs	Fournisseurs	Référentiel, évaluation, historique.
logistique	Logistique	Entrepôts, livraisons, transport.
documents	Documents	Classement, partage, versions.
rapports	Rapports	Rapports métier, filtres, exports.

5.1 Module Commerce

Fonctionnalité	Dépendances ou portée
Tableau de bord	Indicateurs commerciaux et chiffre d'affaires.
Ventes & caisse	Clients et produits ; dépendance métier pour la vente.
Produits & stock	Référentiel des produits et disponibilité.
Clients	Fiches clients.
Fournisseurs	Référentiel fournisseurs.
Achats	Fournisseurs et produits.
Dépenses	Suivi des dépenses.
Comptes de caisse	Suivi des comptes de caisse.
Crédit clients	Suivi des crédits accordés aux clients.
Factures & reçus	Facturation.
Retours & avoirs	Dépend de la vente.
Rapports	Dépend de la vente.
Journal d'activité	Historique des actions commerciales.
Équipe & droits	Collaborateurs et rôles commerciaux.
Paramètres	Configuration du module.

5.2 Module Stocks

Sous-fonctionnalité	Capacité
Tableau de bord	Indicateurs de stock, valeur, alertes et ruptures.
Articles	Catalogue produit, SKU, seuils, prix et archivage.
Entrées de stock	Réceptions et mouvements entrants.
Sorties de stock	Sorties, ventes, pertes et ajustements négatifs.
Demandes	Création, mise à jour, statut et suppression des demandes.
Inventaire	Création, lignes et validation des écarts.
Rapports	Synthèses et alertes stock.
Référentiels	Données de référence du module.
Utilisateurs	Visibilité des utilisateurs concernés.
Paramètres	Configuration du module.

Les dépendances de Stocks sont explicites : les entrées, sorties, demandes, inventaires, rapports, référentiels, utilisateurs et paramètres dépendent du catalogue Articles.

5.3 Module Présences

Fonctionnalité	Capacité
Pointage	Arrivée, sortie, début et fin de pause, retard calculé.
Historique	Historique des présences et des modifications.
Rapports	Synthèses de présence.
Données métier	Absence, planning, mission, congé, jour férié et configuration sont supportés par le modèle de données typé.

5.4 Module Contrôle et coordination

Contrôle est un module transverse de pilotage. Il gère les tâches, décisions, événements de domaine et entrées d'audit.

Élément	Valeurs ou comportement
Statuts de tâche	À FAIRE, EN COURS, VALIDÉ, REFUSÉ, TERMINÉ.
Priorités	BASSE, NORMALE, HAUTE, CRITIQUE.
Événements	Création de tâche, changement de statut, approbation accordée/refusée, événement système.
Périmètre	Administrateur MAXIMUS, administrateur entreprise, manager de secteur ou employé assigné.
Traçabilité	Un événement et un audit peuvent être liés à une décision.

5.5 Organisation et accès

- Création de la hiérarchie entreprise, direction, secteur, service et département.
- Désignation des managers et managers de secteur.
- Création et révocation des comptes employés.
- Définition des rôles et des permissions par module et sous-fonctionnalité.
- Limitation du manager à son unité et à ses descendants.
- Comptes employés créés côté serveur et utilisables immédiatement.

6. Frontend MAXIMUS

6.1 Technologies et responsabilités

Couche	Technologie / rôle
UI	React, TypeScript, composants partagés et écrans par domaine.
Build/dev	Vite.
Navigation	Wouter et History API.
Données locales de démo	StoreData et localStorage pour les scénarios frontend existants.
API clients	auth-api, stock-api, presence-api, control-api.
Permissions UI	employee-permissions, module-features, permission-keys.
Thème entreprise	company-theme et variables CSS par entreprise.

6.2 Navigation

Les liens internes passent par un historique réel. Le bouton Retour de l'application, le bouton précédent du navigateur sur PC et le geste retour sur téléphone partagent le même historique. Un accès direct à une page profonde utilise un retour de secours vers le tableau de bord.

6.3 Personnalisation

- Nom, logo, couleurs principales, couleur d'accent et couleur de barre latérale.
- Navigation adaptée aux modules autorisés.
- Sous-rubriques affichées selon les permissions effectives.
- Interface française et libellés métier explicites.

7. Backend Laravel et API

7.1 Runtime

Laravel 12 est utilisé avec PHP 8.2. Le workflow API provisionne la démonstration puis démarre le serveur PHP intégré via server.php afin de conserver DB_CONNECTION=pgsql dans le même processus. Cette contrainte est documentée comme une décision de runtime.

7.2 Routes d'authentification

Méthode	Route	Rôle
GET	/api/healthz	Healthcheck.
POST	/api/auth/login	Connexion avec email et mot de passe.
GET	/api/auth/session	Résolution de la session courante.
POST	/api/auth/logout	Révocation de la session et suppression du cookie.
POST	/api/auth/accounts	Création ou mise à jour d'un compte employé dans le périmètre autorisé.
DELETE	/api/auth/accounts/{employeeId}	Suspension et révocation d'un compte employé.

7.3 Routes Contrôle

Méthode	Route	Capacité
GET	/api/control/bootstrap	Tâches du périmètre autorisé.
POST	/api/control/tasks	Création d'une tâche selon le rôle.
PATCH	/api/control/tasks/{id}/status	Changement de statut avec trace.

7.4 Routes Présences

Méthode	Route	Capacité
GET	/api/presence/bootstrap	Chargement des éléments de l'entreprise.
POST	/api/presence/items	Création d'un élément typé.
PATCH	/api/presence/items/{id}	Modification dans le même périmètre.
DELETE	/api/presence/items/{id}	Suppression et historique.
POST	/api/presence/clock	Pointage et séquence arrivée/pause/sortie.

7.5 Routes Stocks

Groupe	Routes principales
Lecture	GET /api/stock/bootstrap.
Produits	POST/PATCH/DELETE /api/stock/products.
Fournisseurs	POST/PATCH/DELETE /api/stock/suppliers.
Entrepôts	POST/PATCH/DELETE /api/stock/warehouses.
Emplacements	POST /api/stock/warehouses/{warehouseId}/locations et PATCH/DELETE /api/stock/locations/{id}.
Mouvements	POST /api/stock/movements.
Demandes	POST/PATCH/DELETE /api/stock/requests et changement de statut.
Inventaires	POST /api/stock/inventories et validation.

8. Authentification, rôles et permissions

8.1 Authentification

1. Le frontend envoie email et mot de passe à Laravel.
2. Laravel retrouve un compte actif et vérifie le hash script compatible avec l'ancien système.
3. Laravel crée une session serveur persistée et renvoie un cookie HttpOnly.
4. Les appels suivants utilisent le cookie ; l'acteur est résolu côté serveur.
5. La déconnexion supprime la session et oublie le cookie.

8.2 Hiérarchie d'autorisation

```
Entreprise
├── Unité / secteur / service
│   ├── Rôle
│   │   ├── Sous-fonctionnalité du module
│   │   └── Action : voir, créer, modifier, valider, supprimer, exporter...
```

Un manager de secteur agit dans son unité et ses descendants. Un employé agit dans ses tâches et son périmètre. Un administrateur entreprise gère son entreprise. Un administrateur MAXIMUS peut administrer le catalogue et choisir explicitement une entreprise pour les opérations de contrôle prévues.

8.3 Activation contre permission

Contrôle	Question
Existence	Le module existe-t-il dans le catalogue global ?
Activation	L'entreprise est-elle autorisée à utiliser ce module ?
Permission	Le rôle de l'acteur possède-t-il l'action demandée ?
Périmètre	La donnée appartient-elle à l'entreprise et à l'unité de l'acteur ?

9. Persistance PostgreSQL et modèle de données

9.1 Tables d'authentification

- auth_users : comptes MAXIMUS, rôle, entreprise, employé, secteurs et statut.
- auth_sessions : sessions serveur et empreintes de tokens.
- Les mots de passe sont stockés sous forme de hash ; ils ne sont jamais stockés dans le frontend.

9.2 Tables Contrôle

- control_tasks : tâches, statut, priorité, assignation, entreprise et module lié.
- control_events : événements de domaine et décisions.
- control_audit_entries : audit des opérations importantes.

9.3 Tables Présences

- presence_items : éléments typés de présence, dates, employé, statut, payload extensible et entreprise.
- Les modifications et pointages écrivent des éléments d'historique.

9.4 Tables Stocks

- stock_products : articles et seuils.
- stock_suppliers : fournisseurs.
- stock_warehouses : entrepôts.
- stock_locations : emplacements.

- stock_balances : quantités par produit et localisation.
- stock_movements : entrées, sorties, ventes, achats, transferts, ajustements, pertes et retours.
- stock_requests : demandes et statuts.
- stock_inventories et stock_inventory_lines : inventaires et écarts.

9.5 Règles de migration

- Migrations non destructives lorsque les tables existent déjà.
- Index et contraintes adaptés au périmètre entreprise.
- Transactions pour les mouvements et opérations multi-tables.
- Tests Feature sur SQLite et validation runtime sur PostgreSQL.

10. Sécurité et isolation multi-entreprise

10.1 Middleware

Middleware	Responsabilité
maximus.auth	Refuse une requête sans session valide et expose l'acteur résolu.
maximus.company	Résout le contexte entreprise depuis l'acteur et refuse un companyId étranger.

10.2 Règle d'isolation

Un identifiant companyId envoyé par le navigateur peut être comparé au contexte, mais ne peut pas sélectionner une autre entreprise. Toute lecture et toute écriture métier doit filtrer par le companyId résolu côté serveur. Une requête sans session reçoit 401 ; une entreprise étrangère reçoit 403.

10.3 Traçabilité

- L'acteur est attribué depuis la session, pas depuis une valeur frontend fiable.
- Les décisions de Contrôle créent des événements et audits liés.
- Les Présences conservent un historique des créations, mises à jour, suppressions et pointages.
- Les actions sensibles doivent rester explicites dans les réponses et les logs.

10.4 Frontière future du commerce externe

Un futur site e-commerce public pourra consommer MAXIMUS comme back-office, mais il devra utiliser des routes publiques séparées. Il ne devra pas exposer les routes bootstrap internes ni fusionner son interface publique avec l'administration MAXIMUS.

11. Méthode de développement d'un nouveau module

11.1 Étape A — Contrat produit

1. Choisir un identifiant stable, indépendant de toute entreprise.
2. Décrire le problème métier résolu.
3. Lister les fonctionnalités et dépendances.
4. Définir les actions et rôles.
5. Définir les états, transitions, erreurs et audits.

11.2 Étape B — Backend

```
laravel/
├── app/Http/Controllers/Api/OrdersController.php
├── app/Models/Order.php
├── app/Support/OrdersAuthorization.php
├── database/migrations/..._create_orders_tables.php
├── routes/orders.php
└── tests/Feature/OrdersTest.php
```

- Créer la migration et les modèles.
- Créer les routes protégées par maximus.auth et maximus.company.
- Résoudre le companyId depuis la requête serveur.
- Garder le contrôleur mince et placer les règles complexes dans un service/support.
- Retourner un JSON stable et documenté.

11.3 Étape C — Frontend

```
artifacts/maximus/src/
├── lib/orders-api.ts
├── pages/orders-module.tsx
├── components/orders/
├── routes/app-routes.tsx
└── lib/module-features.ts / permission-keys.ts
```

- Ajouter le module au catalogue global.
- Afficher les fonctionnalités selon les droits effectifs.
- Appeler l'API Laravel au lieu d'écrire directement dans PostgreSQL.
- Gérer 401, 403, 404 et les états de chargement/erreur.
- Ne jamais considérer le masquage UI comme une protection serveur.

11.4 Étape D — Tests et livraison

- Tester l'accès sans session.
- Tester l'entreprise autorisée et l'entreprise étrangère.
- Tester le rôle interdit et la fonctionnalité désactivée.
- Tester les créations, modifications, validations, suppressions et audits.
- Tester le parcours complet écran → API → permission → PostgreSQL.
- Ajouter la documentation du module et la checklist de livraison.

12. Tests et validation actuels

Validation	Résultat observé
Tests Laravel	18 tests réussis, 82 assertions.
Tests frontend permissions	14 tests réussis.
TypeScript	Compilation sans erreur.
Build frontend	Build Vite réussi avec PORT et BASE_PATH de l'artefact.
Healthcheck API	Disponible.
Isolation Stocks	Sans session 401 ; entreprise étrangère 403.
Isolation Présences	Sans session 401 ; entreprise étrangère 403.
Contrôle	Session obligatoire et périmètre entreprise appliqué.

12.1 Commandes de validation

```
# Backend
cd artifacts/api-server/laravel
php artisan test
find app routes tests -type f -name '*.php' -print0 | xargs -0 -n1 php -l

# Frontend
pnpm --filter @workspace/maximus run typecheck
pnpm --filter @workspace/maximus run test
PORT=19624 BASE_PATH=/maximus pnpm --filter @workspace/maximus run build
```

13. Exploitation et runtime de démonstration

Workflow	Commande / comportement
Frontend	pnpm --filter @workspace/maximus run dev.
API	Provisionnement idempotent puis serveur PHP intégré via server.php.
Base	PostgreSQL pour la démo active.
Express	Présent dans le code historique, non lancé comme backend actif.
Déploiement	Non préparé dans cette phase ; la démo reste l'environnement de référence.

14. Limites connues et prochaines étapes

Sujet	État / prochaine étape
Activation du module côté backend	La prochaine étape consiste à persister l'autorisation module-entreprise et à la vérifier sur chaque route Laravel, même en cas de contournement du frontend.
Catalogue global complet	Le catalogue frontend existe ; sa gestion métier doit être progressivement portée par une source serveur durable.
Commerce externe	Architecture séparée définie ; routes publiques et synchronisation à implémenter ultérieurement.
Production	À préparer après preuve de parité Laravel/PostgreSQL et validation des parcours d'accès.
Données KORA	Autorisées comme fixtures de démonstration, interdites comme dépendance du code générique des modules.

15. Glossaire

Terme	Définition
Module	Outil métier global de MAXIMUS, indépendant des entreprises.
Entreprise / tenant	Organisation qui utilise MAXIMUS avec ses propres données.
Fonctionnalité	Capacité visible ou utilisable à l'intérieur d'un module.
Action	Opération sur une fonctionnalité : voir, créer, modifier, valider, supprimer, exporter, etc.
Acteur	Utilisateur résolu depuis la session serveur.
Périmètre	Entreprise, unité, secteur, descendants et tâches accessibles à l'acteur.
Bootstrap	Réponse initiale permettant de charger les données d'un espace ou module.
Audit	Trace durable d'une action ou décision importante.
Fixture de démo	Donnée créée pour valider le produit, sans constituer une règle métier.

16. Conclusion technique

MAXIMUS dispose d'une base exploitable pour développer des modules métier de manière humaine, répétable et sécurisée. La règle centrale est de séparer le produit global — le catalogue et le code du module — de l'utilisation par chaque entreprise — autorisation, configuration, permissions et données. Toute extension doit conserver cette séparation, appliquer l'isolation serveur et être livrée avec ses tests et sa documentation.